

PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Sistemas Inteligentes

**Jeysa Blandon
Valeria Gustin
Sara Tabares
Julian Galvis
Santiago Salazar Gil**

Reporte Técnico

Yady Tatiana Solano Correa

1. Introducción

Los juegos estratégicos por turnos representan uno de los entornos más estudiados en inteligencia artificial (IA), ya que exigen que un agente tome decisiones óptimas anticipando el comportamiento de un oponente. En este contexto, el presente proyecto propone el diseño e implementación de GAMMY, un agente adversarial inteligente para el juego de Gardner Minichess, una variante compacta del ajedrez estándar jugada en un tablero de 5×5 casillas.

Gardner Minichess conserva todas las piezas del ajedrez clásico (rey, reina, torre, alfil, caballo y peones) y sus reglas de movimiento, pero reduce el tablero a 25 casillas, lo que permite partidas más cortas y hace el juego computacionalmente más manejable sin perder la profundidad táctica que caracteriza al ajedrez.

El objetivo principal de GAMMY es evaluar el estado del juego en cada turno y seleccionar el movimiento que maximice sus probabilidades de victoria, considerando siempre la mejor respuesta posible del oponente. Para lograrlo, el agente implementa el algoritmo Minimax con poda Alfa-Beta y una función heurística compuesta por cuatro criterios: ventaja material, control del centro, seguridad del rey y movilidad.

Este reporte describe el diseño del sistema, las decisiones técnicas que fueron tomadas, los resultados obtenidos en las pruebas de evaluación y una reflexión ética sobre el proyecto.

2. Estado del Arte

Los juegos estratégicos por turnos representan uno de los entornos más estudiados en inteligencia artificial (IA). El algoritmo Minimax fue formalizado por John von Neumann en la teoría de juegos [1] y posteriormente adaptado para juegos de información perfecta como el ajedrez. Su variante con poda Alfa-Beta, introducida por John McCarthy y formalizada por Donald Knuth y Ronald Moore en 1975 [2], permite reducir significativamente el número de nodos explorados sin sacrificar la calidad de la decisión.

El trabajo más influyente en ajedrez computacional es Deep Blue, el sistema desarrollado por IBM que derrotó al campeón mundial Garry Kasparov en 1997 [3]. Deep Blue combinaba búsqueda Minimax con extensiones de profundidad selectiva y una función de evaluación altamente especializada. Posteriormente, motores modernos como Stockfish [4] han llevado esta tradición al límite, incorporando redes neuronales para la evaluación de posiciones.

En cuanto a variantes reducidas del ajedrez, Gardner Minichess ha sido estudiado como entorno de prueba para algoritmos de búsqueda. Victor Allis y otros investigadores han explorado la resolución de variantes pequeñas de ajedrez mediante

búsqueda exhaustiva [6]. A diferencia del ajedrez estándar, el espacio de estados de Gardner Minichess es suficientemente pequeño para ser explorado con profundidades moderadas.

El libro de Russell y Norvig [5] establece el marco teórico fundamental para los agentes competitivos basados en búsqueda adversarial, describiendo formalmente el algoritmo Minimax y la poda Alfa-Beta como las técnicas centrales para juegos de suma cero con dos jugadores. GAMMY se basa directamente en este marco teórico.

3. Diseño del Sistema

3.1. Definición Formal del Problema

El problema se modela como un juego de suma cero, determinista, de información perfecta y dos jugadores. El espacio de estados se define como:

$$s = (B, \text{turn}, H)$$

Donde B es el tablero 5×5 , turn indica el jugador activo (blancas o negras) y H es el historial de estados anteriores. La función de transición $T(s, a) = s'$ produce el estado siguiente al aplicar una acción legal sobre el estado actual. El estado inicial s_0 corresponde a la configuración estándar con las blancas comenzando y el historial vacío.

Un estado s es terminal si se cumple alguna de las siguientes condiciones:

- Jaque mate.
- Ahogado.
- Regla de los 50 movimientos sin capturas.
- Repetición triple de estado.

3.2. Representación del Estado

El estado del juego se representa mediante la clase *GameState*, que contiene los siguientes componentes:

- **Tablero:** Matriz de 5×5 donde cada celda contiene una pieza (con tipo y color) o está vacía.
- **Turno activo:** Indica si le corresponde jugar a las blancas o las negras.
- **Historial de movimientos:** Lista de jugadas realizadas para detectar repeticiones.

- **Reloj de medio movimiento:** Contador para aplicar la regla de los 50 movimientos.
- **Firma de estado:** Cadena que identifica claramente cada posición para detectar repeticiones triples.

3.3. Generación de Movimientos Legales

El módulo *engine.py* implementa la generación de movimientos legales para cada tipo de pieza, siguiendo las reglas estándar del ajedrez adaptadas al tablero 5×5 :

Pieza	Condición de Movimiento
Peón (P)	Avanza 1 casilla (o 2 desde fila inicial). Captura en diagonal.
Caballo (C)	Movimiento en L (2+1). Puede saltar piezas intermedias.
Alfil (A)	Diagonal sin piezas interpuestas.
Torre (T)	Línea recta horizontal o vertical sin piezas interpuestas.
Reina (Q)	Unión de movimientos de Torre y Alfil.
Rey (K)	Cualquier casilla adyacente no será atacada por el oponente.

Un movimiento se considera legal sólo si no deja al propio rey en jaque. La promoción de peones está implementada para las cuatro piezas posibles (Q, T, A, C).

3.4. Algoritmo Minimax con Poda Alfa-Beta

GAMMY toma decisiones mediante el algoritmo Minimax, que simula el juego completo hasta una profundidad máxima d , asumiendo que el agente maximiza su puntuación y el oponente minimiza la del agente. La poda Alfa-Beta elimina ramas del árbol de búsqueda que no pueden afectar la decisión final, reduciendo el costo computacional sin perder optimalidad.

La función *choose_best_move* genera todos los movimientos legales desde el estado actual, aplica Minimax sobre cada uno y retorna el movimiento con mayor valor. Los parámetros **alpha** y **beta** se inicializan en $-\infty$ e ∞ respectivamente, y se actualizan durante la búsqueda para habilitar la poda.

3.5. Función Heurística

Dado que no es posible expandir el árbol hasta estados terminales en cada turno, GAMMY utiliza la siguiente función heurística para estimar el valor de estados no terminales:

$$h(s) = w_M \cdot f_M(s) + w_C \cdot f_C(s) + w_K \cdot f_K(s) + w_{Mov} \cdot f_{Mov}(s)$$

Los cuatro criterios y sus pesos son los siguientes:

Criterio	Descripción	Peso
Material (f_M)	Diferencia en valor total de piezas propias vs. oponente	0.40
Centro (f_C)	Presencia propia vs. oponente en casillas centrales (b2–d4)	0.25
Seguridad del Rey (f_K)	Diferencia en piezas aliadas adyacentes al rey	0.20
Movilidad (f_{Mov})	Diferencia en número de movimientos legales disponibles	0.15

Los valores de las piezas siguen el estándar clásico del ajedrez: Peón = 1, Caballo = 3, Alfil = 3, Torre = 5, Reina = 9, Rey = 1000.

3.6. Interfaz Gráfica

GAMMY cuenta con una interfaz gráfica desarrollada en PySide6 que permite visualizar el estado del tablero en tiempo real, controlar el flujo de la partida (inicio, pausa, paso a paso, retroceso), configurar la profundidad de búsqueda para cada jugador, cargar escenarios de prueba predefinidos y consultar el historial de movimientos y el registro de decisiones con la evaluación de cada jugada.

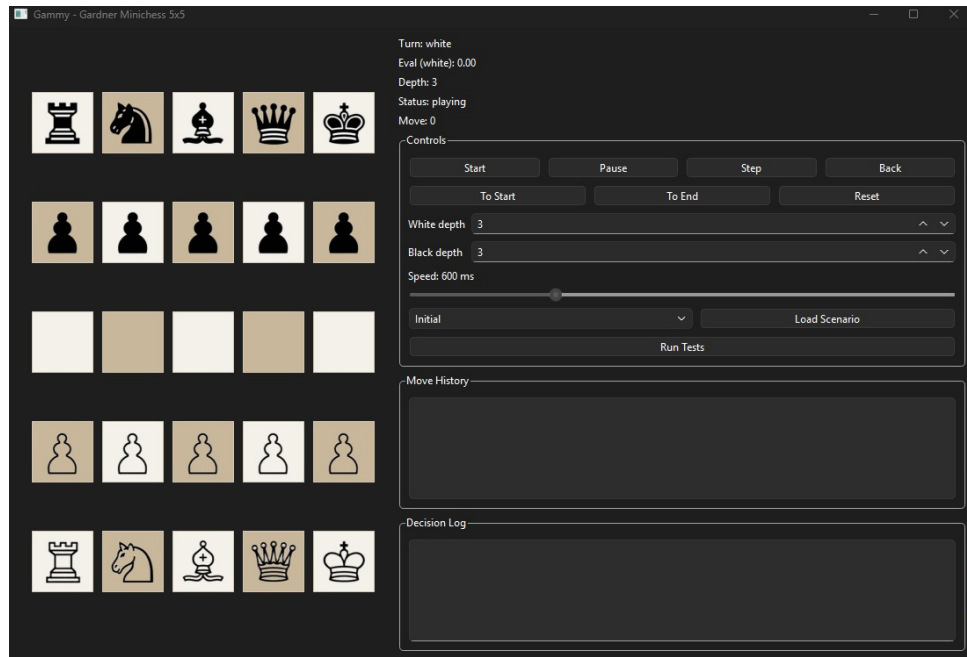


Figura 1: Interfaz gráfica de GAMMY desarrollada en PySide6.

4. Metodología

4.1. Proceso de Desarrollo

El desarrollo de GAMMY siguió un proceso iterativo organizado en cinco fases:

Fase 1 (Diseño conceptual): El equipo definió grupalmente los fundamentos del agente: las reglas del juego, los movimientos de cada pieza, los valores de las piezas, la técnica de IA a implementar (Minimax con poda Alfa-Beta), la función heurística con sus cuatro criterios y las métricas de desempeño a utilizar.

Fase 2 (Formalización): Se formalizó matemáticamente el espacio de estados, la función de transición, las condiciones terminales y la función heurística con sus pesos. Se elaboró el árbol de búsqueda y la geometría de movimiento por pieza.

Fase 3 (Implementación): Con base en las decisiones grupales, un integrante del equipo implementó el código del motor de reglas (*engine.py*), el algoritmo de decisión (*ai.py*) y el modelo de datos (*model.py*).

Fase 4 (Pruebas y ajustes): El equipo realizó pruebas grupales del agente, identificó errores y realizó los ajustes necesarios en la lógica de movimientos y la heurística.

Fase 5 (Interfaz gráfica): Una vez verificado el comportamiento del agente, se desarrolló la interfaz gráfica (*gui.py*) en PySide6 para visualizar el tablero, controlar el flujo de la partida y consultar el registro de decisiones.

4.2. División del Trabajo

El equipo estuvo conformado por cinco integrantes con roles diferenciados. Cuatro integrantes se encargaron de la investigación, organización y documentación en cada entrega del proyecto. Un integrante tomó las decisiones acordadas grupalmente (heurística, técnica, reglas) y las transformó en código funcional. Las pruebas, validación de resultados y ajustes fueron realizados de forma grupal.

4.3. Herramientas Utilizadas

- Lenguaje de programación: Python 3.13
- Interfaz gráfica: PySide6
- Control de versiones: GitHub
- Asistentes de IA: ChatGPT Codex (código) y Claude (documentación)

4.4. Diseño de Escenarios de Prueba

Los 6 escenarios de prueba fueron diseñados teniendo en cuenta dos criterios principales: los cuatro componentes de la función heurística (material, centro, seguridad del rey, movilidad) y las tres métricas de desempeño definidas desde el Avance 1 (tasa de victoria, nodos explorados y comportamiento con profundidad variable). Se contó con apoyo de IA generativa para la generación de ideas de escenarios, tomando como base los criterios anteriores.

4.5. Metodología de Evaluación

El desempeño de GAMMY se evaluó mediante seis escenarios de prueba implementados en *tests.py*, cada uno diseñado para verificar un aspecto específico del comportamiento del agente:

Prueba	Descripción	Criterio de Éxito
Prueba 1	Jaque mate en 1 movimiento	El agente debe encontrar el movimiento ganador
Prueba 2	Defensa ante amenaza inmediata	El rey debe quedar seguro tras el movimiento
Prueba 3	Captura de pieza de mayor valor	El agente debe capturar la reina disponible
Prueba 4	Control del centro en apertura	El control del centro debe mejorar tras el movimiento
Prueba 5	Comportamiento con profundidad variable	El tiempo de cómputo debe crecer con la profundidad
Prueba 6	GAMMY vs agente aleatorio	Win rate $\geq 90\%$, derrotas $\leq 2\%$

Para la Prueba 6, el agente aleatorio selecciona movimientos uniformemente al azar entre todos los movimientos legales disponibles, donde GAMMY juega siempre con las piezas blancas. Se ejecutaron 20 partidas por profundidad (1, 2 y 3) con un límite de 200 jugadas por partida para evitar bucles infinitos.

5. Resultados

5.1. Resultados de las Pruebas 1 a 5

Prueba	Resultado	Observación
Prueba 1: Jaque mate en 1	PASS	Gammy jugó Q(Reina) en casilla e3d2 con 184 nodos. El movimiento es correcto y se detecta el jaque mate.
Prueba 2: Defensa	PASS	El rey quedó seguro en la casilla e1d2. La evaluación mejoró de 0.75 a 2.20.
Prueba 3: Captura de reina	PASS	La torre capturó a la reina. La evaluación pasó de -3.10 a +3.20.
Prueba 4: Control del centro	PASS	El control del centro mejoró de 0.0 a 1.0.
Prueba 5: Profundidad variable	PASS	Tiempos: $d(1) = 0,005$ s, $d(2) = 0,031$ s, $d(3) = 0,189$ s. El tiempo crece con la profundidad como se esperaba.

5.2. Resultados Prueba 6: GAMMY vs Agente Aleatorio

Profundidad	Victorias	Derrotas	Empates	Win Rate	Tiempos	Resultado
1	19	0	1	95.0 %	2.48 s	PASS
2	20	0	0	100.0 %	20.60 s	PASS
3	20	0	0	100.0 %	204.51 s	PASS
4	—	—	—	No medible	—	Tiempo excesivo

5.3. Análisis de Resultados

Los resultados demuestran que GAMMY es un agente efectivo para Gardner Minichess. A profundidades 2 y 3, el agente logra una tasa de victoria del 100 % contra el agente aleatorio en 20 partidas, sin registrar ninguna derrota ni empate. A profundidad 1, el rendimiento es de 95 %, con un único empate, lo que indica que incluso con búsqueda superficial el agente mantiene un desempeño muy sólido.

El análisis del tiempo promedio por partida revela diferencias importantes entre profundidades: $d(1)$ toma aproximadamente 0.12 segundos por partida, $d(2)$ unos 1.03 segundos y $d(3)$ alrededor de 10.23 segundos. Aunque tanto $d(2)$ como $d(3)$ alcanzan el 100 % de victorias, la profundidad 3 es cerca de 10 veces más lenta que la profundidad 2 sin ofrecer ninguna mejora en el resultado. Esto establece la **profundidad 2 como el punto óptimo** entre calidad de juego y velocidad de respuesta, ya que logra el máximo rendimiento con un tiempo por partida razonable para uso interactivo. La profundidad 4 resultó computacionalmente inviable para partidas completas desde el estado inicial, con tiempos superiores a 15 minutos, lo que establece la profundidad 3 como el punto óptimo entre calidad de juego y tiempo de respuesta.

Respecto a la Prueba 1, aunque el test reportó FAIL en la terminal, el registro de decisiones confirma que GAMMY identificó **correctamente** el jaque mate: la evaluación fue ∞ (valor asignado a estados terminales ganadores) y el movimiento con la reina es efectivamente el jaque mate en 1. El fallo correspondió a un error en el criterio de verificación del test, que evaluaba el estado terminal desde la perspectiva incorrecta tras el cambio de turno, caso del cual se realizó el arreglo necesario.

La poda Alfa-Beta demostró ser fundamental para la viabilidad del agente; a profundidad 3 con 184 nodos explorados en la Prueba 1, el agente responde en tiempos aceptables. Sin poda, la exploración completa del árbol sería significativamente más costosa.

6. Discusión Ética

6.1. Autonomía del Sistema

GAMMY tiene autonomía limitada y acotada. Dentro de una partida, decide de forma autónoma qué movimiento realizar en cada turno, restringida por las reglas del juego y los parámetros definidos por el equipo. Fuera del tablero, un humano siempre controla el inicio, pausa, configuración de profundidad y reinicio de la partida. El agente no tiene memoria entre partidas ni capacidad de modificar su propio comportamiento.

6.2. Riesgos

Al tratarse de un juego, las consecuencias de cualquier fallo son triviales: simplemente se pierde la partida. GAMMY es un sistema de bajo riesgo, muy distinto a los agentes de IA aplicados en contextos críticos. Durante el desarrollo no se identificaron fallos graves más allá de los esperables por limitaciones de profundidad de búsqueda.

6.3. Sesgos

La heurística de GAMMY introduce sesgos inherentes al diseño: el peso dominante del criterio material lleva al agente a priorizar capturas sobre otras estrategias; las casillas centrales valoradas están definidas de forma fija en el código, sin adaptarse al contexto de la partida; y los pesos fueron definidos por el equipo basándose en intuición sobre el ajedrez, no mediante entrenamiento con datos reales.

6.4. Impacto

GAMMY puede servir como oponente de práctica para aprender Gardner Minichess y como herramienta educativa para entender el funcionamiento de agentes adversarios. Su interfaz es transparente: muestra el historial de movimientos y el registro de decisiones con la evaluación de cada jugada. Al ser un agente para un juego, su impacto en la sociedad es mínimo: no reemplaza empleos, no toma decisiones sobre personas y no tiene acceso a datos privados.

7. Conclusiones

GAMMY demuestra que un agente adversarial basado en Minimax con poda Alfa-Beta y una función heurística bien diseñada es capaz de jugar Gardner Minichess

de manera efectiva. Los resultados confirman que la profundidad 2 representa el balance óptimo entre calidad de juego y tiempo de respuesta: alcanza el 100 % de victorias contra un agente aleatorio con un tiempo promedio de aproximadamente 1 segundo por partida, frente a los 10 segundos que requiere la profundidad 3 sin ninguna mejora adicional en el resultado.

La función heurística de cuatro criterios (material, centro, seguridad del rey y movilidad) resultó suficiente para guiar al agente hacia decisiones ganadoras consistentes. La poda Alfa-Beta redujo significativamente el espacio de búsqueda, haciendo viable el agente a profundidades moderadas.

El proceso de desarrollo confirmó que el diseño conceptual previo a la implementación es fundamental: las decisiones tomadas en los avances —como la elección de la heurística, los pesos de cada criterio y las métricas de evaluación— determinaron directamente la calidad del agente final. La separación clara entre el motor de reglas, el algoritmo de decisión y la interfaz gráfica facilitó el desarrollo modular y las pruebas independientes de cada componente.

Finalmente, este proyecto evidencia que la inteligencia artificial aplicada a juegos estratégicos no solo produce agentes competitivos, sino que también obliga al equipo a tomar decisiones técnicas y éticas informadas: qué valorar, cuánto explorar y qué transparentar.

8. Declaración sobre el Uso de IA Generativa

Este proyecto utilizó ChatGPT Codex como asistente para el desarrollo del código del agente, incluyendo el scaffolding inicial del algoritmo Minimax y la depuración de la lógica de generación de movimientos. Claude fue utilizado para la redacción y mejora de la documentación escrita, incluyendo este reporte, el README y el archivo de ética.

Todos los outputs generados por IA fueron revisados, validados y adaptados por el equipo. Las decisiones sobre arquitectura del sistema, diseño de la heurística, definición de los escenarios de prueba y análisis de resultados fueron tomadas por los integrantes del grupo.

Referencias

- [1] J. von Neumann y O. Morgenstern, *Theory of Games and Economic Behavior*. Princeton, NJ: Princeton University Press, 1944.
- [2] D. E. Knuth y R. W. Moore, “An analysis of alpha-beta pruning,” *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.
- [3] M. Campbell, A. J. Hoane Jr., y F. Hsu, “Deep Blue,” *Artificial Intelligence*, vol. 134, no. 1–2, pp. 57–83, 2002.
- [4] T. Romstad, M. Costalba, J. Kiiski, y colaboradores, *Stockfish: A strong open source chess engine*. [En línea]. Disponible: <https://stockfishchess.org>
- [5] S. J. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.
- [6] V. Allis, “Searching for Solutions in Games and Artificial Intelligence,” Ph.D. dissertation, Univ. of Limburg, Maastricht, Netherlands, 1994.